

Intermediate (temperature and cost, side by side)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Part 1 — temperature. Prompt used: "give me a one-sentence tagline for a coffee shop."

- At `temperature=0`, all 3 answers came back almost the same, maybe with one word swapped ("Wake up and smell the coffee" vs. "Wake up, smell the coffee"). The model keeps picking its single most-confident next word each time.
- At `temperature=1`, the 3 answers were noticeably different taglines — different angles, different wording, sometimes different length. The model is willing to pick a less-obvious next word, and that choice snowballs into a different sentence.

Part 2 — cost. Using example rates of \$0.15 per 1,000,000 input tokens and \$0.60 per 1,000,000 output tokens:

```
input: 2000 tokens × $0.15 / 1,000,000 = $0.0003
output: 500 tokens × $0.60 / 1,000,000 = $0.0003
total: about $0.0006 for one call
```

Notice output costs the same total here despite using a quarter of the tokens — that's the "output is priced higher per token" idea from Doc03's Core Concepts showing up in real numbers.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1 — temperature, explained mechanically. At each step, the model computes a probability distribution over its whole vocabulary for "what comes next." `temperature=0` collapses that into effectively always taking the single highest-probability token — this is called greedy decoding, and it's why repeated calls converge on nearly identical output. `temperature=1` samples from the distribution closer to its natural shape, so tokens with real but lower probability get picked sometimes, and one different token early in the sentence compounds into a different sentence by the end, since each next token is chosen conditioned on everything before it.

This is also why a fact-lookup or classification prompt barely moves with temperature — the model's distribution is already sharply peaked on one answer regardless of setting, so there's nothing for temperature to meaningfully redistribute. An open-ended creative prompt has a much flatter distribution across many reasonable continuations, so temperature has real material to work with.

Part 2 — cost, with units made explicit.

```
input_rate_per_million  = $0.15
output_rate_per_million = $0.60

input_tokens  = 2000
output_tokens = 500

cost = (2000 / 1_000_000) * 0.15 + (500 / 1_000_000) * 0.60
      = 0.0003 + 0.0003
      = $0.0006
```

Scaling this up matters more than the tiny absolute number suggests: at 10,000 calls a day with this shape, that's \$6/day, or about \$180/month, from one feature alone — and that's before a growing conversation history multiplies the input-token count on every later turn.

Difference from Basic: Basic shows the observed behavior at 2 temperature settings and one cost calculation. Intermediate explains *why* temperature behaves that way (the shape of the probability distribution over the vocabulary, and why it depends on the task) and turns the single cost number into a monthly-scale figure — the difference between "I ran one experiment" and "I understand the mechanism and its real-world cost impact."

Which one should you actually write? For a quick one-off decision (should this one prompt use temperature 0 or 1?), the Basic Version's gut-check question is genuinely enough — don't over-engineer a single prompt. Once you're estimating cost for a real feature, do the Intermediate

Version's monthly-scale math — a rough mental estimate is fine for a prototype, but a real budget conversation with a team needs a number you actually worked out, not one from a notepad.